

LAPORAN PRAKTIKUM
MODUL 11: MEMORI XINU



Disusun Oleh:

Dhiemas Tulus Ikhsan

2311104046 S1SE-07-02

Dosen Pengampu:

Yudha Islami Sulistya, S.Kom., M.Kom

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

JURNAL

1. Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

```
#include <xinu.h>

// Nama: Dhiemas Tulus Ikhsan
// NIM: 2311104046

shellcmd xsh_freememory(int nargs, char *args[]) {

    struct memblk *blk;
    uint32 totalFreeSpace = 0;

    kprintf("Free List:\n");
    kprintf("Block address Length (dec) Length (hex)\n");
    kprintf("-----\n");

    blk = memlist.mnext;

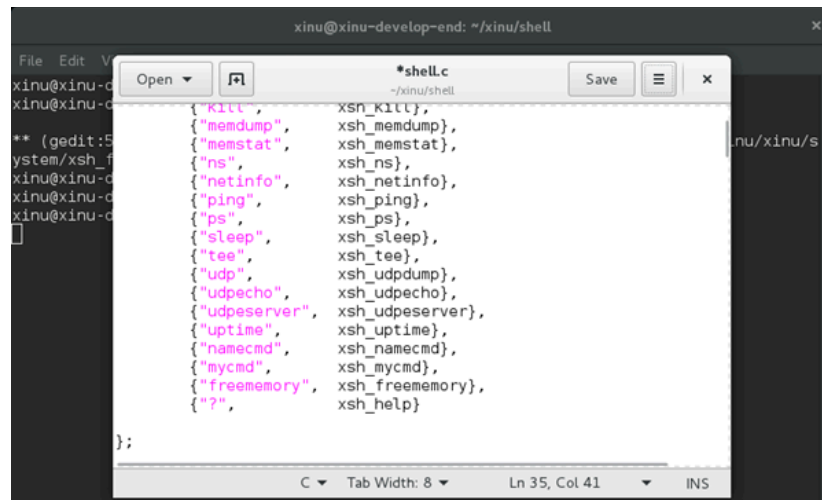
    while (blk != NULL) {
        kprintf("0x%08X %10d 0x%08X\n", (uint32) blk, blk->mlength, blk-
>mlength);

        totalFreeSpace += blk->mlength;
        blk = blk->mnext;
    }

    kprintf("\nTotal FreeSpace %d\n", totalFreeSpace);

    return 0;|
```

Edit shell.c



Edit shprototypes.h

```
/* in file xsh_freememory.c */
extern shellcmd xsh_freememory(int nargs, char *args[]);
```

```
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help

-----DHIEMAS TULUS IKHSAN-----
XINU
-----2311104046-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018E110     -975128      0xFFFF11EE8
0x00100000      5070048      0x004D6000
0x005E8000       57344        0x0000E000

Total FreeSpace 4153064
xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

2. Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Pemisahan antara Data Segment dan BSS Segment didasarkan pada status inisialisasi variabel global yang dikandungnya; Data Segment mengalokasikan variabel yang telah diinisialisasi, sedangkan BSS Segment diperuntukkan bagi variabel yang belum memiliki nilai awal. Struktur pemisahan ini mengoptimalkan efisiensi memori karena ukuran BSS Segment tidak perlu dieksplisitkan secara penuh di dalam berkas biner program executable.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Selama sistem operasi Xinu beroperasi, kapasitas free space pada memori bersifat dinamis. Perubahan ini terjadi karena aktivitas alokasi memori oleh suatu proses akan mereduksi ruang yang tersedia, sedangkan proses dealokasi akan mengembalikan memori tersebut ke sistem sehingga meningkatkan kembali kapasitas free space.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Dari aspek manajemen memori, heap memiliki risiko kerentanan yang lebih tinggi akibat mekanisme pengelolaannya yang dilakukan secara manual, sehingga berpotensi memicu memory leak atau fragmentasi jika terjadi kesalahan penanganan. Sebaliknya, stack menawarkan tingkat keamanan yang lebih terjamin karena dialokasikan secara otomatis menggunakan struktur data LIFO (Last-In, First-Out).

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Struktur data linked list diterapkan untuk mengoptimalkan efisiensi sistem

dalam mengelola blok memori kosong berukuran variatif. Lewat mekanisme ini, proses pencarian, penyimpanan, penambahan, hingga penggabungan ruang memori terfragmentasi dapat dilakukan secara dinamis selama sistem berjalan.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Efisiensi penggunaan memori pada heap sering kali terkendala oleh beberapa tantangan utama, seperti risiko kebocoran memori (memory leak), isu fragmentasi, serta kompleksitas manajemen manualnya. Karakteristik ini membuat kontrol terhadap heap jauh lebih rumit dan rentan dibandingkan dengan otomatisasi yang ada pada stack.